



# Abstraction Layers & Hibernate

---

March 2026

# Recap days 1 & 2



## Day 1

JDBC, connections, queries, object mapping, DAO pattern, DataSources

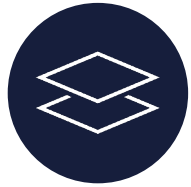


## Day 2

Transactions, ACID, isolation levels, optimistic & pessimistic locking



We started to build a mini-ORM framework with annotations, generic mapping, and transaction support



## Today

Abstraction layers & Hibernate

# Module overview



## Day 1

JDBC & Object  
Mapping



## Day 2

Transactions &  
Concurrency



## Day 3

Abstraction Layers  
& Hibernate



## Day 4

Entity Relations &  
Fetch Strategies



## Day 5

ORM Caching &  
Performance

# Time planning



Starting time: 9.30



Breaks: ...



End time: 16.30





# Section 1

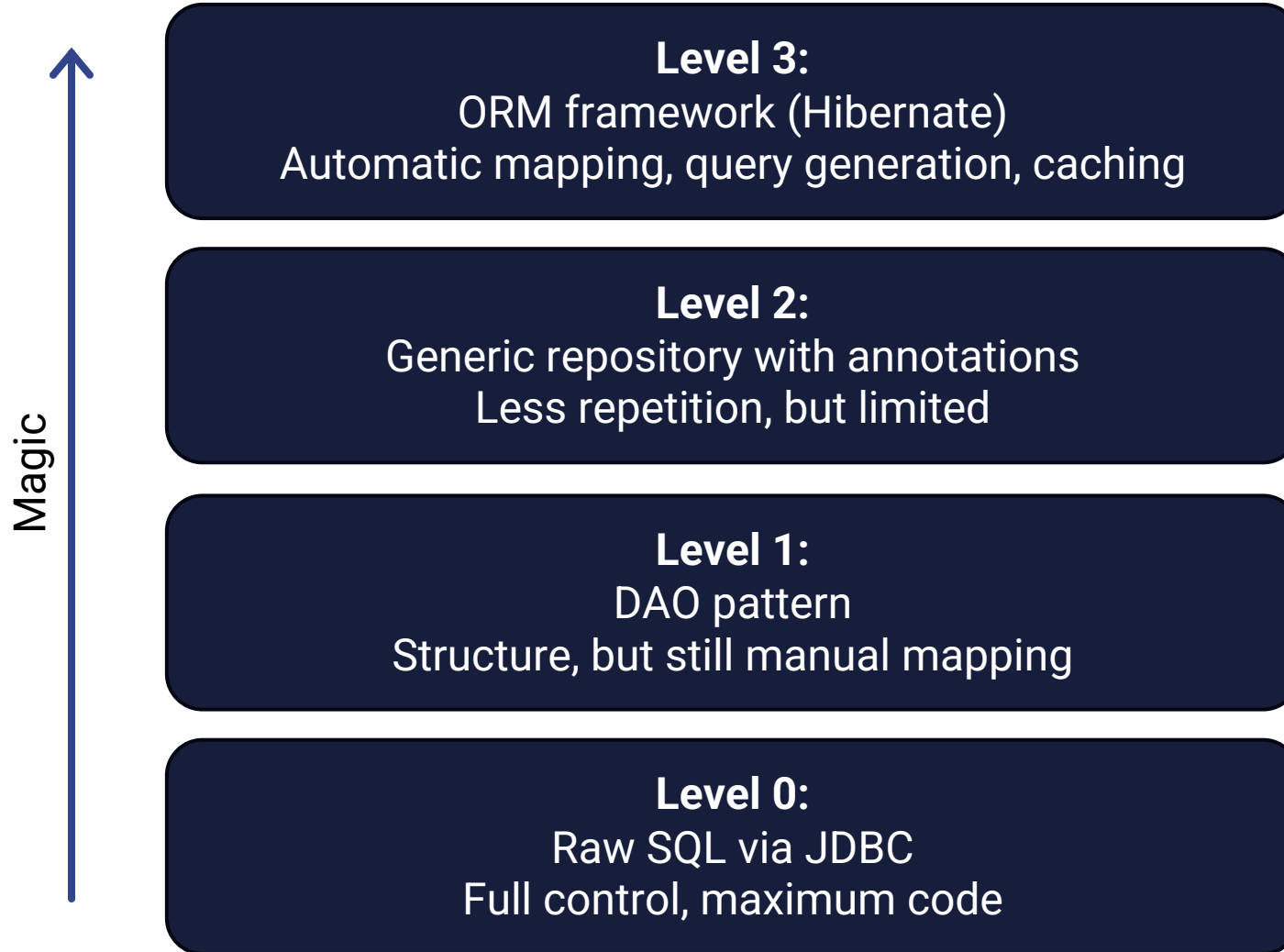
From JDBC to abstraction - why ORM?



# What we built

- **Day 1:** raw JDBC (full control, lots of code)
- **Day 2:** transactions, locking (even more moving parts)
- **Our mini-ORM:** annotations, generic mapping, repositories
- What stood out?
  - Lots of boilerplate
  - Lots of repetition
  - Easy to make mistakes

# The path of abstraction





# What is an abstraction layer?

- A layer that hides complexity and provides a simpler interface
- Analogy: driving a car
  - You turn the steering wheel, press the gas pedal
  - You don't need to know how the engine, transmission, or fuel injection work
  - But: when the car breaks down, you need to understand what's underneath to fix it



# The trade-off of abstraction



**Advantages:** productivity, readability, fewer bugs



**Disadvantages:** less control, “magical” behavior, performance surprises, bugs can be more complicated



**Leaky abstractions:** the underlying layer “leaks” through



Joel Spolsky's Law of Leaky abstractions:

*“All non-trivial abstractions, to some degree, are leaky”*



# Why Hibernate?

- The most widely used ORM framework in the Java world
- Implements the JPA (Jakarta Persistence API) standard
- Not focussing on learning it as a "black box" — understanding how it works
- Comparison with your own framework:
  - Where does Hibernate solve the same problems?
  - Where does it go further?





# Section 2

**Hibernate**

Introduction & entity mapping

# What is Hibernate?



An ORM (Object-Relational Mapping) framework



Implements the JPA specification



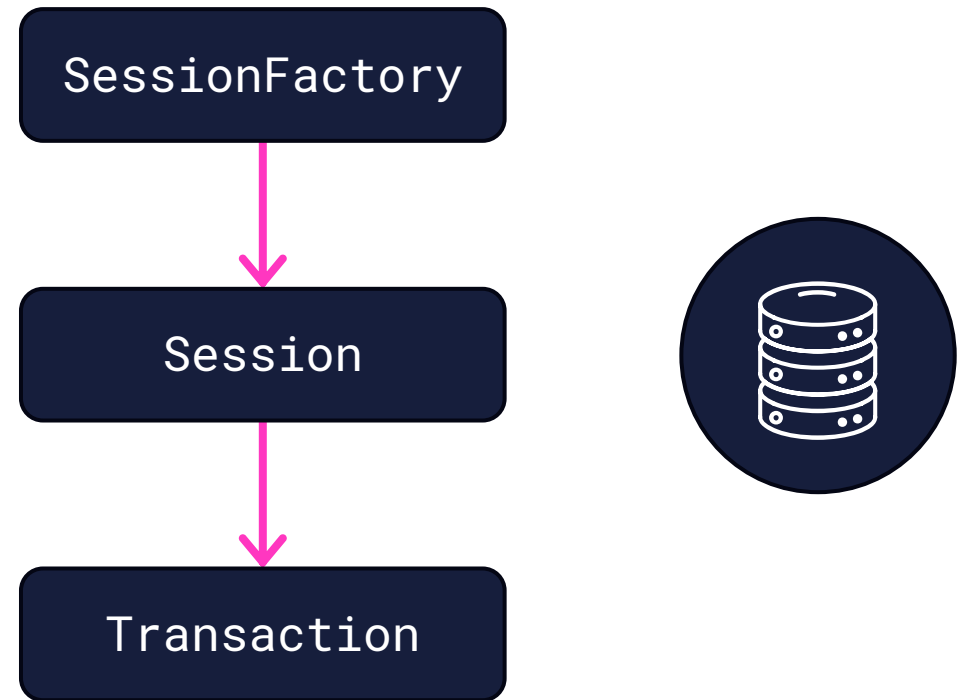
Maps Java objects to database tables automatically



Manages the full lifecycle of entities

# Hibernate architecture (high level)

- **SessionFactory** – the factory that creates Sessions (heavyweight, one per application)
- **Session** – the workspace for database operations (lightweight, one per request/thread)
- **Transaction** – transaction management within a Session
- Comparison: **SessionFactory**  $\approx$  your **ConnectionManager** + metadata setup



# The SessionFactory and the startup process



At startup, Hibernate scans all `@Entity` classes



It builds an internal metamodel: tables, columns, relationships



This is a one-time, expensive operation



**Important:** if you change an entity while Hibernate is running, Hibernate doesn't know



You must restart to pick up entity changes



Analogy: printing a map at the start of a road trip. If the road changes during the trip, your map is wrong



# Entity mapping with annotations

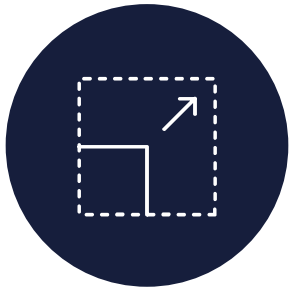
- `@Entity`
  - Marks a class as a database entity
- `@Table(name = "...")`
  - Links to a specific table name
- `@Id`
  - The primary key field
- `@GeneratedValue`
  - Automatic ID generation (IDENTITY, SEQUENCE, AUTO)
- `@Column(name = "...")`
  - Links a field to a specific column
- **Default mapping:** if you don't specify `@Column`, Hibernate uses the field name



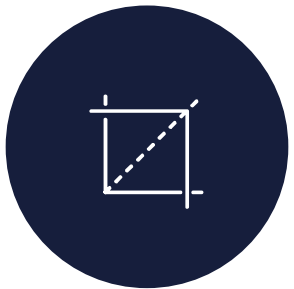
# Comparison with your own framework



You built similar annotations: `@Entity`, `@Table`, `@Column`, `@Id`



Hibernate does the same thing, but with many more options and edge cases



Your framework is simpler but more limited (that's fine, the goal was understanding)

# Demo

## Hibernate Project Setup and Entity Mapping

- A Maven project with Hibernate and H2 dependencies
- Configure `hibernate.cfg.xml` or `persistence.xml` or do this in code for now
- Create a `Student` entity with Hibernate annotations
- Start the `SessionFactory` and let Hibernate create the table
- Compare: in our framework, we wrote DDL ourselves; Hibernate generates it



# Mini exercise

## Hibernate entity mapping

- Use the provided Hibernate + H2 starter project
- Create a `Course` entity with annotations: `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`
- Decide which fields need explicit `@Column(name = "...")` and which can use default mapping
- Start the `SessionFactory` and verify the `courses` table was created automatically
- Compare: how much code did you need for the same result on day 1?







# Section 3

CRUD & JPQL in Hibernate

# Create: persisting objects



`session.persist(student)` saves a new object to the database



Hibernate generates the INSERT SQL automatically



The ID is filled in automatically after persist



You don't write any SQL

# Read: Retrieving objects



`session.get(Student.class, id)` loads an object by primary key



Hibernate generates the SELECT and maps the result to an object

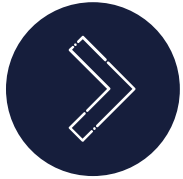


Returns `null` if the object doesn't exist



Alternative: `session.find(Student.class, id)` (JPA standard)

# Update: dirty checking



Change a field on a persistent object: `student.setName("New Name")`



Hibernate detects the change automatically. This is called "dirty checking"



At commit time, Hibernate generates the UPDATE SQL



You do NOT need to call "save" or "update" explicitly



This is the "magic" (and it can surprise you)



# Delete: removing objects



`session.remove(student)` marks the object for deletion



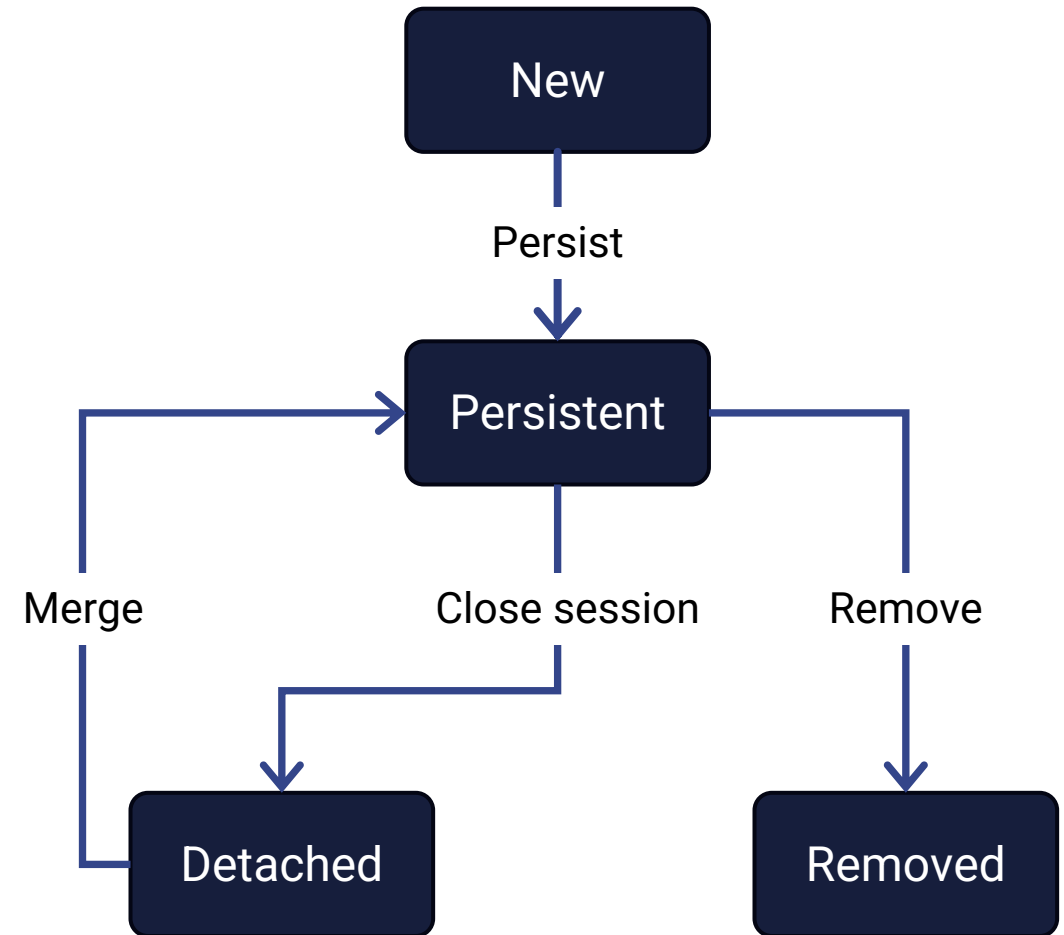
Hibernate generates the DELETE SQL at commit time



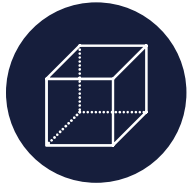
Alternative: `session.delete(student)`  
(Hibernate-specific, older API)

# Entity lifecycle states

- **Transient:** new object, not connected to a Session
- **Persistent:** connected to a Session, changes are tracked
- **Detached:** was persistent, but the Session is closed
- **Removed:** marked for deletion
- Many Hibernate "bugs" are caused by misunderstanding these states



# JPQL - Java Persistence Query Language



SQL-like syntax, but works with objects instead of tables



`SELECT s FROM Student s WHERE s.age > :age` instead of  
`SELECT * FROM students WHERE age > ?`



Named parameters: `:age` instead of `?` → more readable



Database-independent: works with your entity model

# JPQL example



```
List<Student> students = session.createQuery(  
    "SELECT s FROM Student s WHERE s.age > :minAge", Student.class)  
    .setParameter("minAge", 20)  
    .getResultList();
```

# JPQL vs SQL

| JPQL                         | SQL                              |
|------------------------------|----------------------------------|
| Works on entities and fields | Works on tables and columns      |
| SELECT s FROM Student s      | SELECT * FROM students           |
| s.age                        | age (column name)                |
| Database-independent         | May use database-specific syntax |
| Named parameters: :minAge    | Positional parameters: ?         |



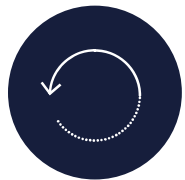
# Transactions in Hibernate



`session.beginTransaction()` starts a transaction



`transaction.commit()` commits



`transaction.rollback()` rolls back



Comparison: your `TransactionManager` does the same thing, but Hibernate integrates it into the Session



Hibernate expects you to always work within a transaction for write operations

# Demo

## CRUD and JPQL in action

- Add a student with `session.persist()`
- Retrieve the student with `session.get()`
- Modify the name. Hibernate detects the change at commit (dirty checking)
- Delete a student
- Execute a JPQL query to filter students by age
- Show the generated SQL (`hibernate.show_sql=true`)



# Mini exercise: CRUD & JPQL

- Use the `Course` entity from the previous exercise (or the demo `Student` entity)
- **Create:** persist 3 courses
- **Read:** retrieve one course with `session.get()`
- **Update:** change a course name (without calling `save()`) and commit → observe dirty checking
- **Delete:** remove one course
- Turn on `hibernate.show_sql=true` and examine the generated SQL for each operation
- Write a JPQL query: find all courses with more than 3 credits
- **Experiment:** modify a field on a persistent object and commit. Does Hibernate generate an UPDATE without you asking?
- **Bonus question:** what happens if you modify a field on a detached object (outside the Session)? Try it and explain the result



## Section 4

The limits of abstraction: polymorphism & proxies



# All abstractions leak



Hibernate makes life easier in 80% of the cases



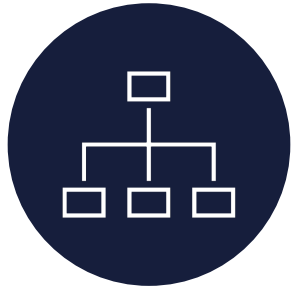
The other 20% can be surprisingly hard



That's where production issues arise, so it's exactly that 20% you need to understand.



# The impedance mismatch: inheritance



Java: `Student extends Person`, `Teacher extends Person` → class hierarchy



Database: tables, rows, columns → no concept of inheritance



How do you store an inheritance hierarchy in tables that don't support it?

# Strategy 1: single table

- Everything in one table with a discriminator column
- The discriminator column tells Hibernate which subtype each row is
- Pro: simple queries, good performance
- Con: lots of **NULL** columns for fields that don't belong to every subtype

**Persons**

| <b>Id</b> | <b>Name</b>  | <b>Type (discriminator)</b> | <b>Student_number</b> | <b>Department</b> |
|-----------|--------------|-----------------------------|-----------------------|-------------------|
| 1         | Polly Morph  | Student                     | 314159                | <b>NULL</b>       |
| 2         | Hugh Bernate | Teacher                     | <b>NULL</b>           | Science           |

## Strategy 2: table per class (joined)

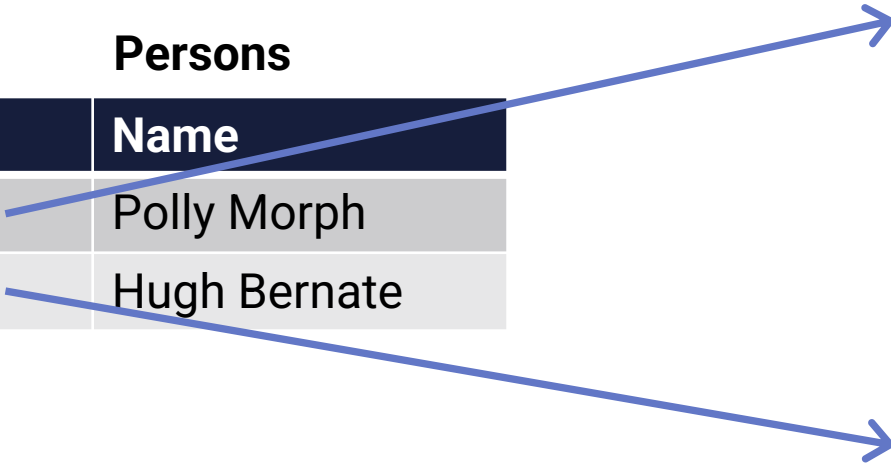
- Each class gets its own table
- **persons** table: id, name
- **students** table: person\_id (FK), student\_number
- **teachers** table: person\_id (FK), department
- To load a Student: JOIN **persons** and **students**
- **Pro:** normalized, no NULL waste
- **Con:** complex JOIN queries, slower with deep hierarchies

# Strategy 2: table per class (joined)

| Persons |              |
|---------|--------------|
| Id      | Name         |
| 1       | Polly Morph  |
| 2       | Hugh Bernate |

| Students       |                |
|----------------|----------------|
| Person_id (FK) | Student_number |
| 1              | 314159         |

| Teachers       |            |
|----------------|------------|
| Person_id (FK) | Department |
| 2              | Science    |



# Strategy 3: Table per concrete class

- Each concrete class gets a complete table
- **students** table: id, name, student\_number (all fields, including inherited ones)
- **teachers** table: id, name, department (all fields, including inherited ones)
- No **persons** table at all
- **Pro:** simple queries per subtype
- **Con:** data redundancy, UNION queries for polymorphic queries

Students

| Id | Name        | Student_number |
|----|-------------|----------------|
| 1  | Polly Morph | 314159         |

Teachers

| Id | Name         | Department |
|----|--------------|------------|
| 1  | Hugh Bernate | Science    |



# No perfect strategy

|                     | Single table     | Table per class (joined) | Table per concrete class |
|---------------------|------------------|--------------------------|--------------------------|
| Queries per subtype | Fast (one table) | Slower (JOINS)           | Fast (one table)         |
| Polymorphic queries | Fast             | Moderate (JOINS)         | Slow (UNIONS)            |
| NULL columns        | Many             | None                     | None                     |
| Redundancy          | None             | None                     | High                     |

# The supertype list problem



You want to retrieve a `List<Person>` from the database



The list contains both `Student` and `Teacher` objects



Hibernate must determine the correct subtype for each row



With Single Table: the discriminator column tells Hibernate the type → straightforward



With Table Per Class: multiple JOINs needed → less efficient



Calling a subtype-specific method? → Casting needed

# Proxies: what are they?

- Hibernate uses **lazy-loaded proxies**: stand-in objects that load data only when needed
- A proxy is a subclass of your entity, generated by Hibernate at runtime
- It looks like your entity, but it's actually a `Person$HibernateProxy`
- This is an implementation detail that leaks through the abstraction



# The proxy problem with casting

- `person instanceof Student` can return `false` even if the object IS a Student
- Why? The proxy is a `Person$HibernateProxy`, not a `Student`
- Hibernate doesn't know which subtype it is until the data is loaded
- This breaks normal Java `instanceof` checks

Java expects:



Hibernate reality:



# The solution: unproxying



`Hibernate.unproxy(person)` returns the real underlying object



`Hibernate.getClass(person)` returns the real entity class



After unproxying, `instanceof` works correctly

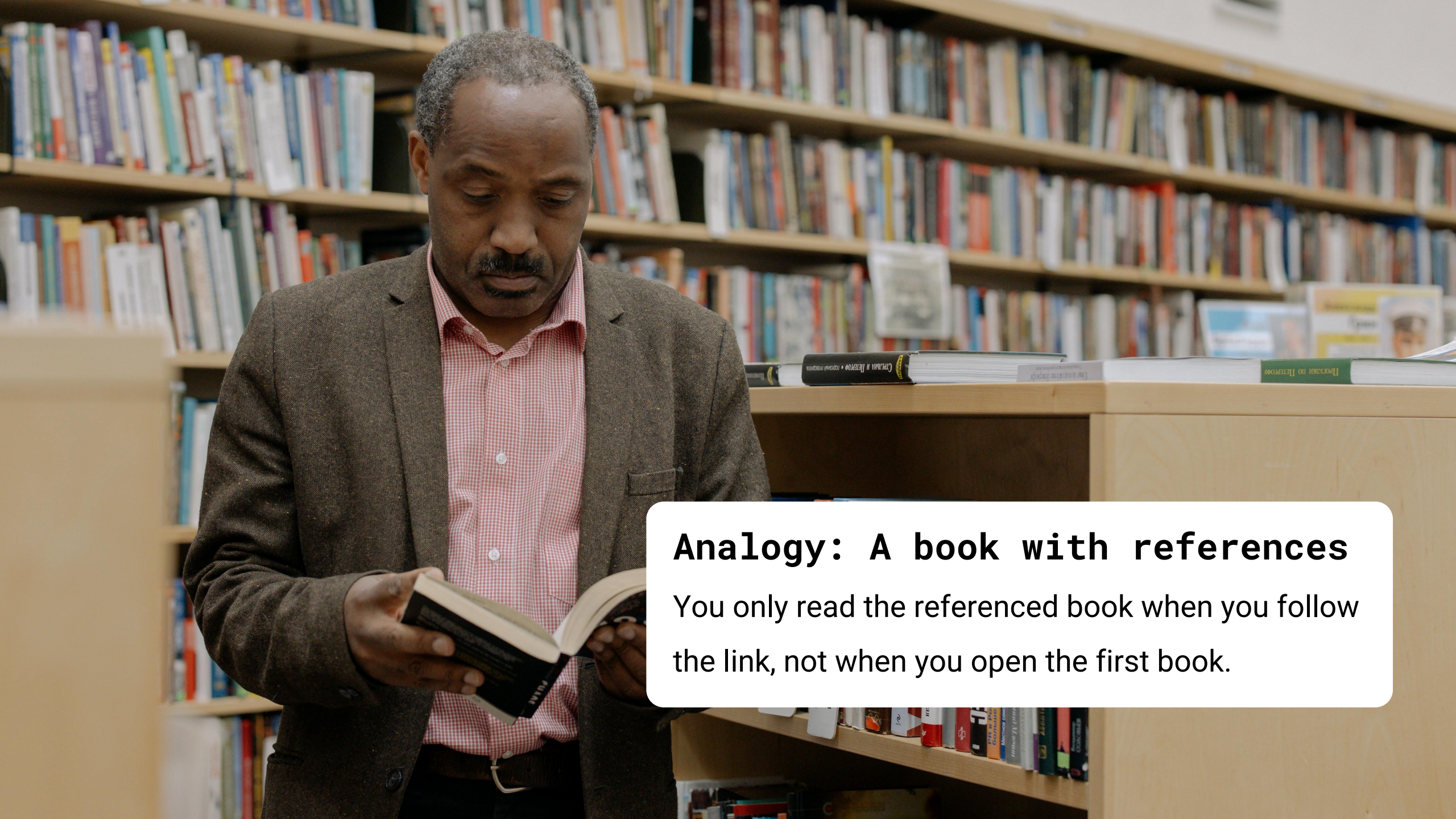


Always unproxy before doing type checks on Hibernate entities

# Lazy loading: a first introduction

- By default, Hibernate loads related collections only when you access them (lazy)
- `department.getTeachers()` triggers a SELECT at that moment
- But: if the Session is already closed → `LazyInitializationException`
- Why lazy? Performance: you don't want to load the entire database into memory
- More on this in day 4. For now, just know it exists





## **Analogy: A book with references**

You only read the referenced book when you follow the link, not when you open the first book.



# Demo: Polymorphism and Proxies in Hibernate

- Create a `Person` superclass with `Student` and `Teacher` subtypes (Single Table)
- Persist a mix of students and teachers
- Retrieve a `List<Person>` and show Hibernate returns the correct subtypes
- Demonstrate the proxy problem:
- Load a `Person` lazily via `session.getReference()`
- Show that `instanceof Student` fails on the proxy
- Fix it with `Hibernate.unproxy()`
- Show a `LazyInitializationException` by accessing a collection outside the Session

# Mini exercise: polymorphism & proxies

- Create a `Course` superclass with `@Inheritance(strategy = InheritanceType.SINGLE_TABLE)` and a `@DiscriminatorColumn`
- Create `OnlineCourse` and `ClassroomCourse` as subtypes, each with their own field (e.g., `platformUrl` for online, `roomNumber` for classroom)
- Persist 2 online courses and 2 classroom courses, retrieve them as `List<Course>`
- Check with `instanceof` whether the correct subtypes are returned
- Provoke the proxy problem: load a `Course` via `session.getReference()` (lazy proxy) and try `instanceof OnlineCourse` → observe the result
- Fix it with `Hibernate.unproxy()` and verify that `instanceof` now works correctly
- **Bonus:** retrieve a `Department` with a lazy `@OneToMany` collection of courses, close the Session, call `getCourses()` → catch the `LazyInitializationException` and think about how you would prevent this



# Section 5

SessionFactory, debugging & Hibernate in practice

# The SessionFactory and the metamodel



Hibernate builds the metamodel at startup



This includes: all entities, their fields, relationships, table structure, query plans



It's a one-time, expensive operation: the SessionFactory is a singleton



Consequence: runtime changes to entities are NOT picked up





# Common debugging pitfall

- Scenario: you're debugging, you change a `@Column` annotation or add a field
- You don't restart, but Hibernate is still using the old metamodel
- Result: your change has no effect, or you get cryptic errors
- Cause: the `SessionFactory` was built before your change
- Solution: **always restart after changing an entity**



# Hibernate configuration options you should know

- `hibernate.show_sql=true` >> show generated SQL (essential for debugging!)
- `hibernate.format_sql=true` >> format the SQL for readability
- `hibernate.hbm2ddl.auto` >> schema generation:
  - `create`: create tables on every start (data lost!)
  - `update`: adjust schema to match entities (handy for development)
  - `validate`: check that schemas match (for production)
  - `none`: do nothing
- **Warning:** `update` is NOT for production. Use migration tools (Flyway, Liquibase)

# When Hibernate and when not?



Hibernate excels at: standard CRUD, simple relationships, rapid prototyping



Hibernate struggles with: complex reporting queries, bulk data processing, exotic data models



Knowing when to reach for JDBC directly is an important skill

# Discussion: experiences with abstraction boundaries

- When did you feel the limits of abstraction in your own framework?
- Which problems that Hibernate solves did you also encounter?
- Where would you rather keep control (JDBC) vs. delegate (Hibernate)?

# Morning wrap-up

## What we've covered today

- The path from JDBC to Hibernate: each layer adds abstraction
- How Hibernate handles entity mapping, CRUD, and JPQL
- The limits: polymorphism, proxies, casting problems
- Lazy loading as a first concept introduction (more on day 4)
- The SessionFactory and the importance of restarting after entity changes



# This afternoon: extending your own framework



Add batch operations and polymorphism support to your own framework



Explore the limits of your own abstraction



Compare your solutions with how Hibernate handles the same problems